

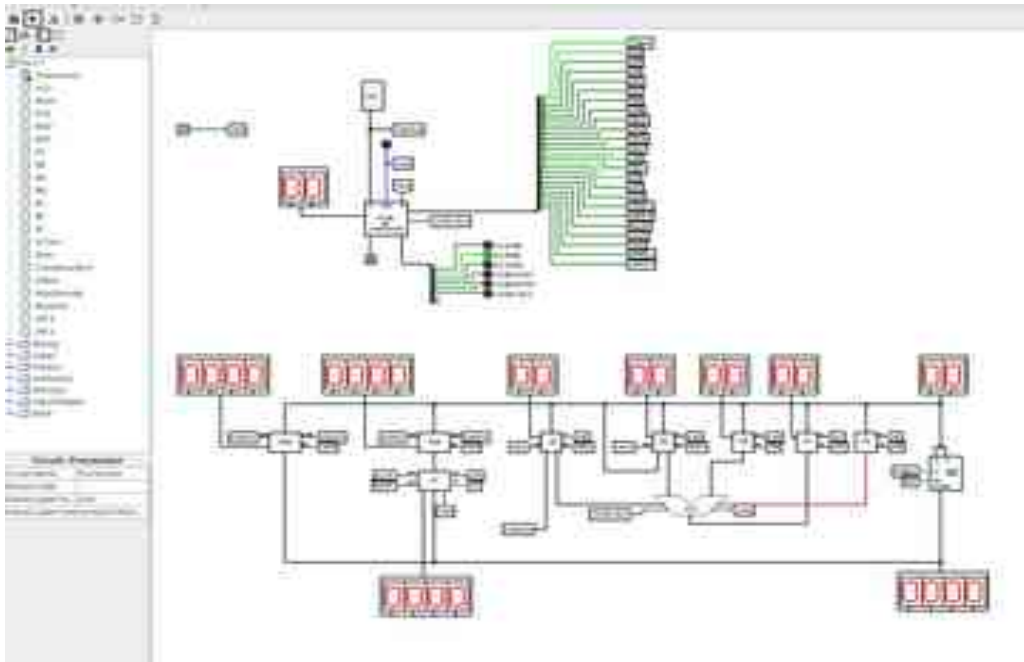
Structure des ordinateurs – TP – Programmation du processeur PIsO

Dans ce TP vous allez écrire et exécuter plusieurs programmes sur le processeur PIsO qui tourne sous Logisim.

1. La machine

Vous devez utiliser la version originale de Logisim (<https://www.cburch.com/logisim/>) ou le fork de LogisimIta (<https://github.com/Logisim-Ita/Logisim>)

Télécharger la machine sur e-uapv et ouvrez-la avec Logisim :



On retrouve tous les composants de la machine :

- la mémoire,
- l'ALU
- les registres : PC, RAP, RAD, RI, RA, RB, RC et RE.
- l'unité de commande avec tous les signaux qu'elle envoie pour piloter la machine.

Plusieurs afficheurs sont positionnés pour observer les contenus des registres, des bus et de la mémoire.

Pour utiliser la machine il faut :

1. remplir la ROM avec les séquençements des instructions,
2. charger un programme en mémoire RAM,
3. vérifier que PC contient l'adresse de la 1^{re} instruction
4. activer l'horloge,
 - manuellement en cliquant sur le composant ou avec le menu **Simulate/Tick Once (Ctrl-T)**
 - automatiquement avec le menu **Simulate/Ticks Enabled (Ctrl-K)** et dans ce cas on peut choisir la fréquence (**Simulate/tick Frequency**)

2. Remplissage de la ROM

Télécharger le fichier **ROM-PIso1**, chaque ligne de ce fichier contient une ligne de la ROM.

Il contient :

- le **fetch** est en 1^{re} ligne,
- le **decode** en 2^e ligne,
- l'instruction de code 1 en 16^e ligne, **pourquoi 16 ?**
- le séquençement de toutes les instructions, sauf l'instruction de code 2.

1 Compléter l'instruction de code 2.

Télécharger le fichier **Aide-Remplissage-ROM.ods**, il indique l'ordre des signaux dans la ROM et calcule les valeurs hexadécimales correspondantes.

En utilisant ce fichier, ou pas :

- vérifiez rapidement les séquençements proposés,
- complétez l'instruction de code 2 qui a été effacée !

3. Interaction ROM-séquenceur

1 Chargement de la ROM

Après avoir complété le fichier ROM-Plso1-sans-instruction2, sauvez-le sous le nom ROM-Plso1 et chargez-le dans la ROM :

- ouvrir le circuit **Rom**,
- clic droit sur le contenu de la ROM et **Load Image** (ou **Edit Contents** si vous voulez le modifier à la main !)

2 Le séquenceur (Unité de commande)

Ouvrez le séquenceur (**UCom**), on retrouve les deux multiplexeurs :

- multiplexeur **Mux Decode** qui réalise le décodage de l'instruction en sélectionnant l'adresse dans la ROM des signaux à envoyer suivant le code de l'instruction à exécuter,
- multiplexeur **Mux Etat** qui intervient dans les instructions conditionnelles en sélectionnant la fin de l'instruction en fonction du résultat du test.

Les adresses en entrée des 2 multiplexeurs sont toutes à 0, remplacez-les par les bonnes valeurs.

Indices :

- **Mux Decode** : regardez la ROM
- **Mux Etat** : regardez le séquençement des lignes 10 et 11

4. Exécuter un programme

Attention pour la suite :

- pour accéder à la mémoire il faut double-cliquer sur le composant mémoire de la machine et pas dans le menu de gauche, Logisim est parfois capricieux et ne prend pas toujours les modifications faites sinon,
- un reset de la machine efface la mémoire et il faut la remplir ou la charger à nouveau.

La machine est maintenant prête à exécuter un programme, voici quelques exemples de programme.

1 Affectation simple

Soit une variable V à l'adresse 0100 l'instruction pour lui affecter la valeur $(10)_{10}$ est :

V := 0A

et son codage est :

10 01 00 0A

pour l'exécuter il faut la placer en mémoire, à l'adresse 0 qui sera le contenu de PC au démarrage et activer le nombre de tops d'horloge nécessaires. Commencez par un **reset** de la machine : **Simulate/Reset Simulation** ou Ctrl-R.

Placez cette instruction dans la mémoire et exécuter-là :

- sélectionnez le circuit **Mem**, clic droit puis **Edit Contents** sur la mémoire et entrez les 4 octets à partir de l'adresse 0000,
- sélectionnez le circuit processeur et activer l'horloge le nombre de top nécessaires (**Ctrl-T**) pour exécuter le

- programme, vous pouvez garder la fenêtre **Hex Editor** ouverte,
- à la fin de l'exécution, la case 0100 doit contenir 0a !

2 Enchaînement d'instruction

Chargez le fichier **Prog1-affectation.exe** et éditez-le. Les lignes qui commencent par # sont des commentaires, le programme contient 4 instructions d'affectation et une dernière instruction Vers qui fait boucler le programme sur cette dernière instruction. C'est une manière d'arrêter la machine !

Faites un **reset** de la machine, régler la vitesse de l'horloge à 1 Hz avec **Simulate/Tick Frequency** pour avoir le temps de voir ce qu'il se passe, chargez le programme en mémoire et exécutez-le par **Simulate/Ticks Enabled (Ctrl-K)** qui démarre l'horloge.

Surveillez les valeurs des variables en regardant la mémoire et attendez que le programme boucle sur les adresses 16-17-18.

3 Décoder un petit programme

Chargez le fichier **Prog2.exe** et exécutez-le. Vous pouvez accélérer la machine en augmentant sa fréquence. Pendant qu'il tourne, essayez comprendre ce qu'il fait et donnez le contenu des 2 variables à la fin.

4 Écrire un programme « à la main »

Copiez le fichier **Prog2.exe** en **Prog3.exe** et modifiez-le pour en faire un programme qui calcule X^n .

5 Écrire un programme avec un assembleur

L'écriture de programme directement en code machine et en hexadécimal n'est pas facile. Heureusement on peut écrire des programmes qui vont traduire les instructions du langage lisible en code machine. On appelle ce langage, langage d'assemblage ou par abus assembleur, ainsi que le programme qui se charge de la traduction en code machine.

Récupérez le programme **asm-Piso**, qui est un assembleur pour le processeur PIsO. Il s'agit d'un exécutable pour les machines du CERI, si vous voulez le recompiler pour un ordinateur différent vous pouvez récupérer les sources.

Un assembleur est un "vrai" langage de programmation et comme souvent il impose quelques règles d'écriture pour les programmes, sa documentation est disponible sur la page du TP : "Petite documentation de l'assembleur PIsO".

Charger le fichier **Prog4.piso**, écrit en assembleur PIsO, et l'exécuter après l'avoir assemblé bien sûr.

Écrire et exécuter les programmes suivants :

1. maximum de 3 variables,
2. calcul de $N!$,
3. racine d'une équation du 1^{er} degré,
4. calcul de la racine carrée d'un entier par somme des impairs successifs :
(<https://mathesis-online.com/algorithmes-calcul-racines/>)
5. racine d'une équation du 2nd degré,
6. etc.